VIET NAM NATIONAL UNIVERSITY HO CHI MINH CITY
INTERNATIONAL UNIVERSITY

Fundamental Concepts of Data Security

W5 - One-Way Hash Function

Le Hai Duong, PhD. (lhduong@hcmiu.edu.vn)

Overview

One-way hash function, also known as **cryptographic hash** function, is an essential building block in cryptography.

- generates a number of a fixed size (called hash) from a message of an arbitrary length.
- useful to many applications
  - password authentication,
  - integrity preservation,
  - to the blockchain technology and Bitcoin

Outline

- Concepts and properties of one-way hash function
- Applications of one-way hash functions
- Message Authentication Code (MAC)
- Blockchain and Bitcoins
- Hash collision attacks

Number guessing game

1. Lecturer and student, each picks 1 number
2. Students show their number
3. Lecturer shows his/her number
4. If the sum of all number is
    a. ODD ➜ students win
    b. EVEN ➜ lecturer wins

Concepts and Properties

Cryptographic properties

hash function is any function that can be used to map data of an **arbitrary size** to data of a **fixed size**, and needs to satisfy two important properties:

- *One-way property:* Given a hash value `h`, it should be difficult to find any message `m`, such that `hash(m) = h`. Obviously, there are many messages that can generate the same hash value `h` because a hash function has a many-to-one mapping; however, it should be difficult to find any such message for a given `h`.

- *Collision-Resistance property:* It should be difficult to find two messages m_1 and m_2, such that $\texttt{hash}(m_1) = \texttt{hash}(m_2)$.

```
f(x) = x mod 1000
```

👈 a hash function ???

Replay Number guessing game

1. Lecturer picks a number, calculate its hash value H(n), and publish the hash to the class
2. Students picks their number and show them to the class
3. Lecturer reveals his/her number
4. Everyone calculate the hash H(n) to verify lecturer's number
5. If the hash is valid and it is
    a. ODD ➜ students win
    b. EVEN ➜ lecturer wins

1. What makes the protocol fair for the lecturer?
2. What makes the protocol fair for the lecturer?
3. A potential attack???

Algorithms and Programs

MD series

SHA series

...

The MD (Message Digest) series

- MD2 ← broken & obsolete
- MD4 ← broken & obsolete
- MD5 ← collision resistance property was broken in 2004, but one-way property has not
  → not good for digital signature
- MD6 ← developed by Ron Rivest and his team, candidate for SHA-3 but lost; not widely adopted yet

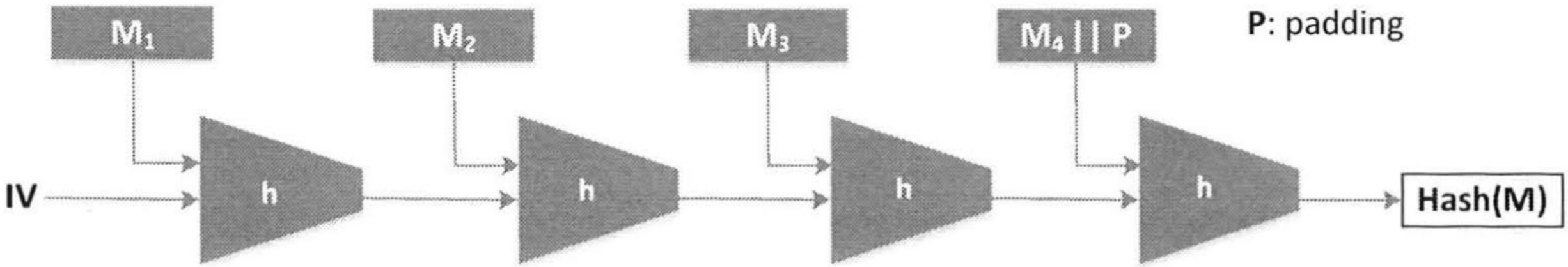
The SHA (Secure Hash Algorithm) Series

Secure Hash Algorithms are a family of cryptographic hash functions published by the National Institute of Standards and Technology (NIST).

- SHA-0 ← withdrawn shortly after publication due to an undisclosed "significant flaw"
- SHA-1 ← 160-bit hash function was designed by the National Security Agency (NSA); considered as a weak hash function since 2005, and was recommended not to be used because collision-resistance property broken
- SHA-2 ← designed by the NSA (SHA-256 and SHA-512)
- SHA-3 ← released by NIST on August 5, 2015 after chose Keccak as SHA-3; different construction structure than the one used by SHA-1 and SHA-2 (also by MD5)

How Hash Algorithm Works

Merkle-Damgard construction



What distinguish the MD5, SHA- I, and SHA-2 hash functions are their different compression functions, block sizes, and hash sizes.

One-Way Hash Commands

```
seed@VM:~/.../Lab2$ md5sum plaintext
6165796b89ff6ddeb242beafcf567dcf  plaintext
seed@VM:~/.../Lab2$ sha256sum plaintext
b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60
plaintext
seed@VM:~/.../Lab2$ echo -n "6165796b89ff6ddeb242beafcf567dcf" | wc
      0       1      32
seed@VM:~/.../Lab2$ echo -n
"b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60" |
wc
      0       1      64
seed@VM:~/.../Lab2$ echo "64 * 4" | bc
256
```

```
seed@VM:~/.../Lab2$ openssl dgst --list
Supported digests:
-blake2b512               -blake2s256               -md4
-md5                      -md5-sha1                 -mdc2
-ripemd                   -ripemd160                -rmd160
-sha1                     -sha224                   -sha256
-sha3-224                 -sha3-256                 -sha3-384
-sha3-512                 -sha384                   -sha512
-sha512-224               -sha512-256               -shake128
-shake256                 -sm3                      -ssl3-md5
-ssl3-sha1                -whirlpool
seed@VM:~/.../Lab2$ openssl dgst -sha256 plaintext
SHA256(plaintext)= b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60
seed@VM:~/.../Lab2$ sha256sum plaintext
b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60  plaintext
seed@VM:~/.../Lab2$ openssl sha256 plaintext
SHA256(plaintext)= b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60

seed@VM:~/.../Lab2$ md5sum plaintext
6165796b89ff6ddeb242beafcf567dcf  plaintext
seed@VM:~/.../Lab2$ openssl md5 plaintext
MD5(plaintext)= 6165796b89ff6ddeb242beafcf567dcf
seed@VM:~/.../Lab2$ openssl dgst -md5 plaintext
MD5(plaintext)= 6165796b89ff6ddeb242beafcf567dcf
```

Check the hash

```
seed@VM:~/.../Lab2$ openssl dgst -md5 plaintext > plaintext.md5
seed@VM:~/.../Lab2$ openssl dgst -sha256  plaintext > plaintext.sha256
seed@VM:~/.../Lab2$ md5sum -c plaintext.md5
plaintext: OK
seed@VM:~/.../Lab2$ sha256sum -c plaintext.sha256
plaintext: OK

seed@VM:~/.../Lab2$ md5sum plaintext
6165796b89ff6ddeb242beafcf567dcf  plaintext
seed@VM:~/.../Lab2$ echo -n "6165796b89ff6ddeb242beafcf567dcf
plaintext" | md5sum -c
plaintext: OK
```

Computing one-way hash in programs

```
// Calculate SHA-256 hash in SQL programs
$ mysql
mysql> SELECT SHA2('message', 256);
+------------------------------------------------------------------+
| SHA2('message', 256)                                             |
+------------------------------------------------------------------+
| ab530a13e45914982b79f9b7e3fba994cfd1f3fb22f71cea1afbf02b460c6d1d |
+------------------------------------------------------------------+
```

```
// Calculate SHA-256 hash in Python
$ python
>>> import hashlib
>>> m = hashlib.sha256()
>>> m.update("message")
>>> m.hexdigest()
'ab530a13e45914982b79f9b7e3fba994cfd1f3fb22f71cea1afbf02b460c6d1d'
```

```
// Calculate SHA-256 hash in PHP
$ php -a
php > echo hash('sha256', 'message');
ab530a13e45914982b79f9b7e3fba994cfd1f3fb22f71cea1afbf02b460c6d1d
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <openssl/sha.h>

void main()
{
  SHA256_CTX ctx;
  u_int8_t results[SHA256_DIGEST_LENGTH];
  int i;
  char *msg_part1 = "Part One ";
  char *msg_part2 = "Part Two ";
  char *msg_part3 = "Part Three";

  SHA256_Init(&ctx);                          ①
  SHA256_Update(&ctx, msg_part1, strlen(msg_part1));
  SHA256_Update(&ctx, msg_part2, strlen(msg_part2));   ②
  SHA256_Update(&ctx, msg_part3, strlen(msg_part3));
  SHA256_Final(results, &ctx);                ③

  /* Print the message and the hash */
  printf("%s%s%s\n", msg_part1, msg_part2, msg_part3);
  for (i = 0; i < SHA256_DIGEST_LENGTH; i++)
      printf("%02x", results[i]);
  printf("\n");
}
```

Applications of One-Way Hash Functions

Integrity Verification

seed@VM:~/.../Lab2$ **echo -n "Hello world" | sha256sum**
64ec88ca00b268e5ba1a35678a1b5316d212f4f366b2477232534a8aeca37f3c -
seed@VM:~/.../Lab2$ **echo -n "Hfllo world" | sha256sum**
af978301c426a69e0acc5ab28667ac3139d8fdde07d1bf112a2c18c0f901c4b4 -

seed@VM:~/.../Lab2$ **openssl dgst -sha256 plaintext > plaintext.sha256**
seed@VM:~/.../Lab2$ **sha256sum -c plaintext.sha256**
plaintext: OK

seed@VM:~/.../Lab2$ **sha256sum plaintext**
b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60
plaintext
seed@VM:~/.../Lab2$ **echo -n**
"b6ca1009cc6c8d7bfed77672fee88de5d9a9bde49faef5369249d2aa72f81b60
plaintext" | sha256sum -c
plaintext: OK

Thank you for downloading Ubuntu Desktop

[Thank you for downloading Ubuntu Desktop | Ubuntu]

Your download should start automatically. If it doesn't, download now.

You can verify your download, or get help on installing.

Run this command in your terminal in the directory the iso was downloaded to verify the SHA256 checksum:

```
echo "5fdebc435ded46ae99136ca875afc6f05bde217be7dd018e1841924f71db46b5 *ubuntu-20.04.3-desktop-amd64.iso" | shasum -a 256 --check
```

You should get the following output:

```
ubuntu-20.04.3-desktop-amd64.iso: OK
```

Or follow this tutorial to learn how to verify downloads

Help
and u

Community projects

Committing a Secret Without Telling It

- Due to the collision-resistance property, once the hash is published, there is no way to change the original message without being detected.
- We can use these properties to commit a secret without actually disclosing the secret.

Password Verification

seed@VM:~/.../Lab2$ **sudo cat /etc/shadow | grep "\$6" | cut -f1,2 -d":"**
seed:6n8DimvsbIgU0OxbD$YZ0h1EAS4bGKeUIMQvRhhYFvkrmMQZdr/hB.Ofe3KFZQTgFTcRgoIoKZdO0rhDRxxaITL4b/scpdbTfk/nwFd0
os:6Rc5uIngj0wIl5dUD$D.NPgkkTQqohc4Xm1WELZmm0mCNPtTqsEan54Vr3W7IzNMOOgT/j9k0J9X1Vg41pmFe89JMkqFWrNeSS4xpYf0

seed@VM:~/.../Lab2$ **python**
Python 2.7.18 (default, Mar 8 2021, 13:02:45)
[GCC 9.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> **import crypt**
>>> **print(crypt.crypt('dees', '6n8DimvsbIgU0OxbD$'))**
6n8DimvsbIgU0OxbD$YZ0h1EAS4bGKeUIMQvRhhYFvkrmMQZdr/hB.Ofe3KFZQTgFTcRgoIoKZdO0rhDRxxaITL4b/scpdbTfk/nwFd0

Trusted Timestamping

Example: You wrote a 500-page novel and how do you protect against plagiarism?

- Approach 1: Using a printed media
- Approach 2: using a trusted party
- Another approach: using blockchains

Message Authentication Code (MAC)

Man-In-The-Middle (MITM) attack



attacker can intercept the data from the sender, make changes, and send the modified data to receiver

How to prevent this type of attack???

Constructing MAC and Potential Attacks

Trial: Incorporate a secret key in one-way hash functions is to mix the secret with the message, and then hash the mixture

$$MAC(K, M) = H(K \,||\, M)$$

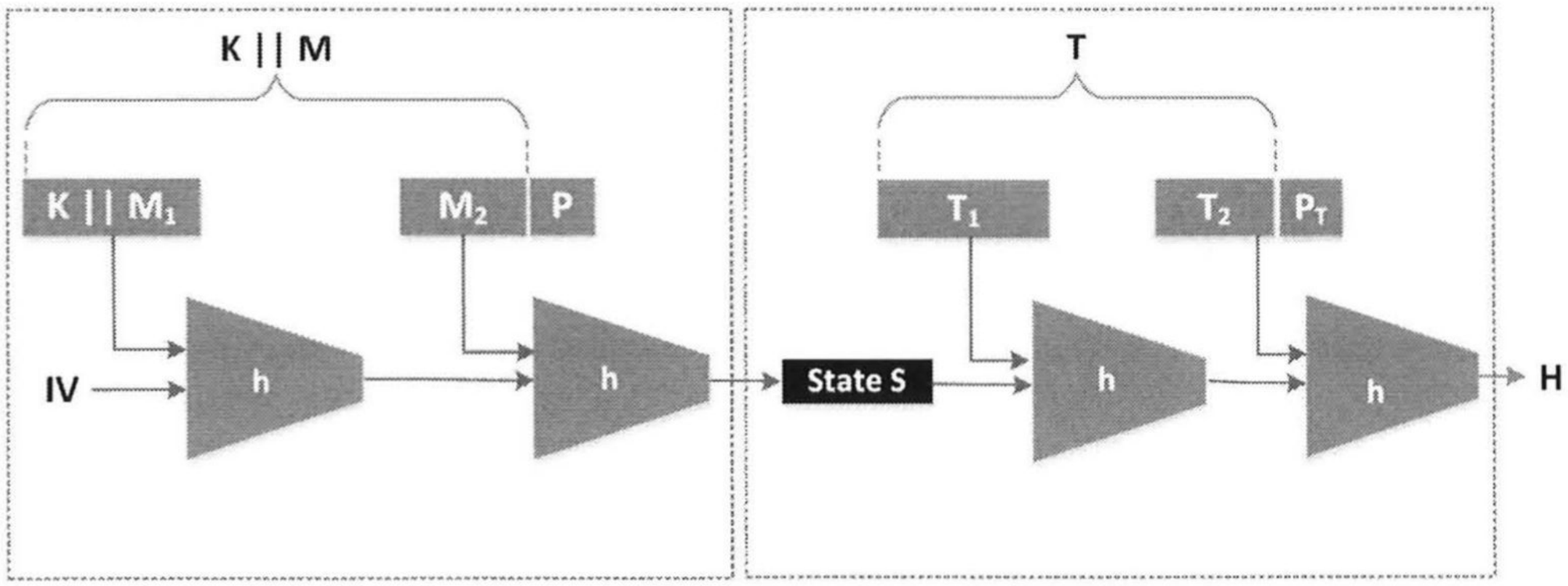
length extension attack

`MAC(K, M) = H(K || M)`

attacker can compute valid

`H(K || M || P || T)`

without knowing key K



S can be derived from Hash(K || M)

H = Hash(K || M || P || T)

Flickr's API Signature Forgery Vulnerability

Thai Duong and Juliano Rizzo

Date Published: Sep. 28, 2009 Advisory ID: MOCB-01

Advisory URL: http://netifera.com/research/flickr_api_signature_forgery.pdf Title: Flickr's API Signature Forgery Vulnerability Remotely Exploitable: Yes

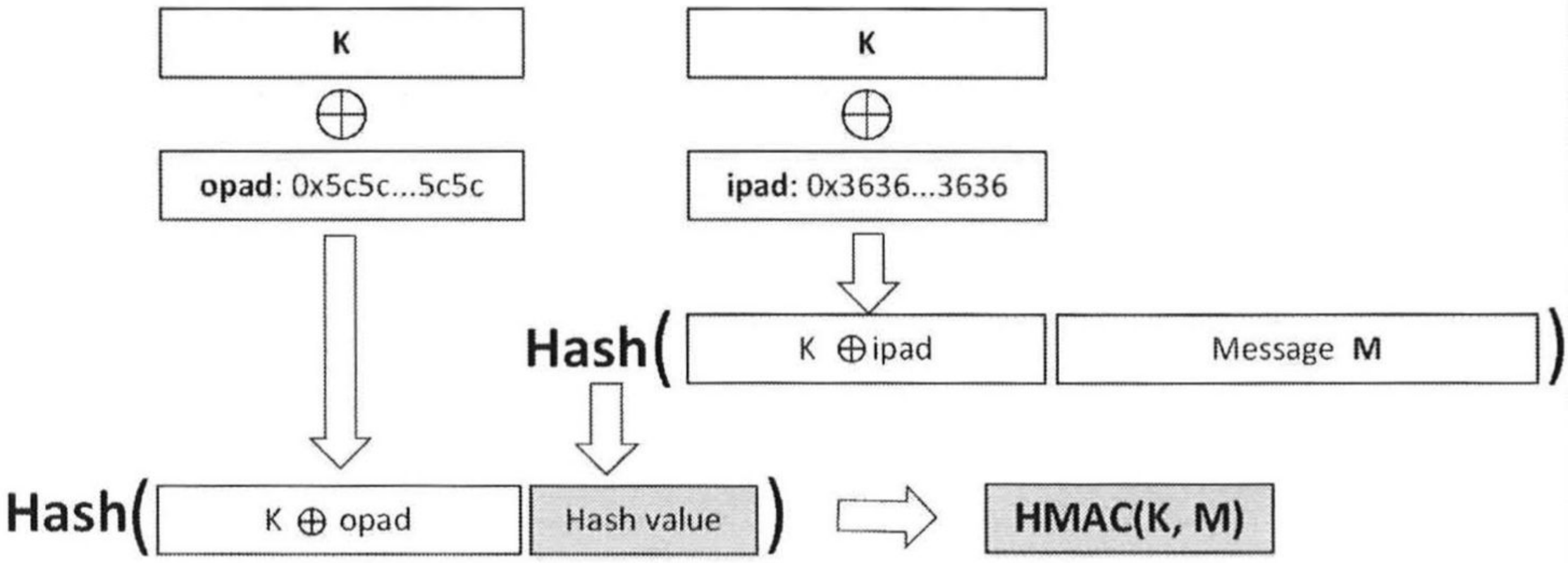
1. Vulnerability Description

Flickr is almost certainly the best online photo management and sharing application in the world. As of June 2009, it claims to host more than 3.6 billion images. In order to allow independent programmers to expand its services, Flickr offers a fairly comprehensive web-service API that allows programmers to create applications that can perform almost any function a user on the Flickr site can do. The Flickr's API consists of a set of callable methods, and some API endpoints. To perform an action using the Flickr's API, you need to select a calling convention, send a request to its endpoint specifying a method and some arguments, and will receive a formatted response. Many methods require the user to be logged in. At present there is only one way to accomplish this. Users should be authenticated using the Flickr Authentication API. Any applications wishing to use the Flickr Authentication API must have already obtained a Flickr's API Key. An 8-byte long 'shared secret' for the API Key is then issued by Flickr and cannot be changed by the users. This secret is used in the signing process, which is required for all API calls using an authentication token. In addition, calls to the flickr.auth.* methods and login URLs pointing to the auth page on Flickr must also be signed. For more details, please read the Flickr Authentication API Spec [1]. This advisory describes a vulnerability in the signing process that allows an attacker to generate valid signatures without knowing the shared secret. By exploiting this vulnerability, an attacker can send valid arbitrary requests on behalf of any application using Flickr's API. When combined with other vulnerabilities and attacks, an attacker can gain access to accounts of users who have authorized any third party application. Additionally, if an application uses PHPFlickr >= 1.3.1, an attacker can trick users of that application to visit arbitrary web sites. This may apply for other Flickr's API libraries and applications as well.

Should **not** try to invent our own algorithm to generate hash-based MAC; instead, we should use standard and well-established algorithms.

The Keyed-Hash MAC (HMAC) Algorithm

Keyed-hash message authentication code (HMAC) [Krawczyk et al., 1997) is widely used in various security protocols, including IPSec and TLS protocols.



seed@VM:~/.../Lab2$ **echo -n "My public speech" | openssl dgst -sha256**
(stdin)= 4ae2ea6c10e853533599efb9f92923e80429e97b6d98e9252cd6cec649e73bf7

seed@VM:~/.../Lab2$ **echo -n "My public speech" | openssl dgst -sha256 -hmac "secretkey"**
(stdin)= 22d2d3563b66881361e69460d7400d5603f943262b4c072fa5f12af7a8c8efd6

seed@VM:~/.../Lab2$ **echo -n "My public speech" | openssl dgst -sha256 -hmac "secret"**
(stdin)= 0f52839c33df5ea643d2d574d90f6185373637539789cbb9089858ee744137

Blockchain and Bitcoins

Blockchain

- Blockchain was conceptualized in 2008 by an anonymous person or group known as Satoshi Nakamoto
- It was implemented in 2009 as a core component of Bitcoin
- A blockchain is a continuously growing list of records, called blocks, which are linked and secured using cryptography
- A blockchain is typically managed by ledgers in a peer-to-peer network, not by any central party.
- Once a block is linked and accepted, it is extremely difficult to modify any record inside the block, because that requires alteration of all subsequent blocks on the majority of the ledgers on the peer-to-peer network, i.e., it requires collusion of the network majority.
- Due to these properties, blockchains are being used in many applications. Bitcoin is probably the most successful application of blockchains.

Hash Chain and Blockchain

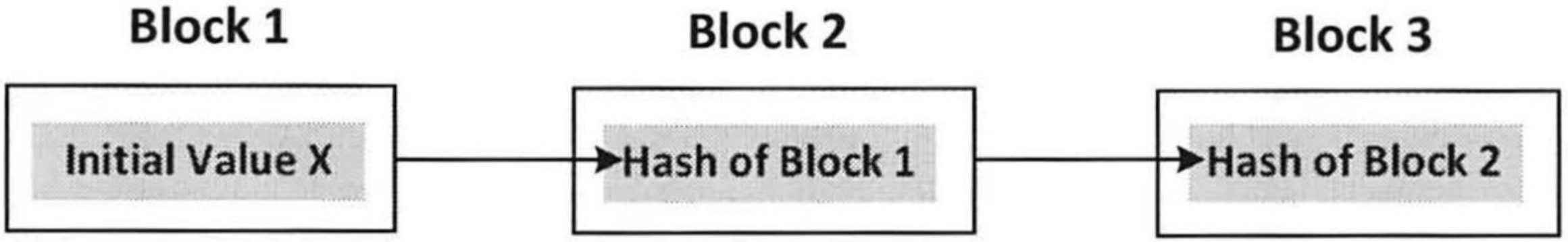


Figure 22.5: Hash chain

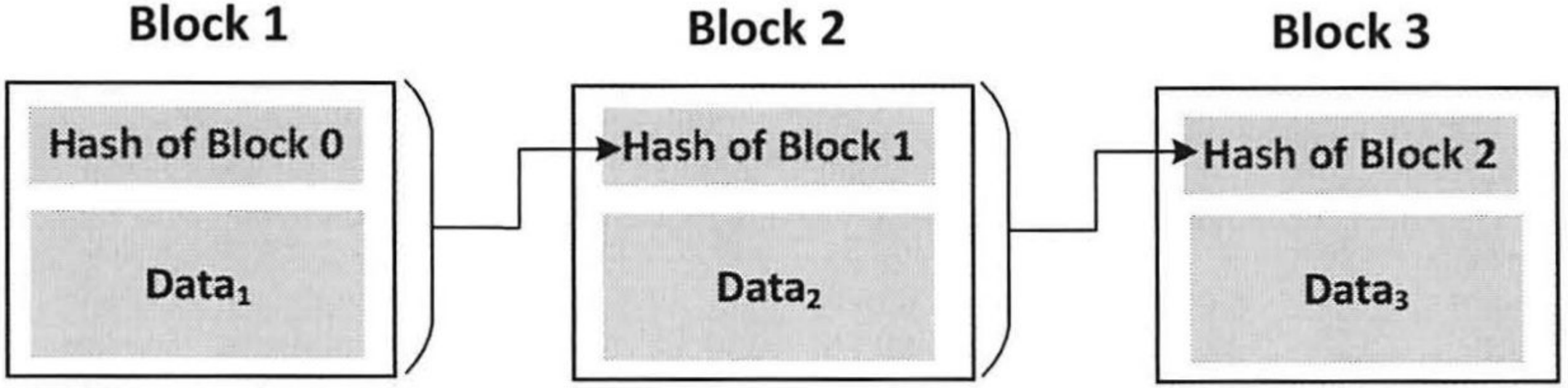


Figure 22.6: Blockchain

Block: 0 | Blockchain Explorer

Make Chaining Difficult

Block 704137

Block: 704137 | Blockchain Explorer

USD BTC

This block was mined on October 09, 2021 at 5:32 AM GMT+7 by Poolin. It currently has 1 confirmations on the Bitcoin blockchain.

The miner(s) of this block earned a total reward of 6.25000000 BTC ($338,448.31). The reward consisted of a base reward of 6.25000000 BTC ($338,448.31) with an additional 0.08780748 BTC ($4,754.93) reward paid as fees of the 2837 transactions which were included in the block. The Block rewards, also known as the Coinbase reward, were sent to this address.

A total of 51,134.09400618 BTC ($2,768,999,652.42) were sent in the block with the average transaction being 18.02400212 BTC ($976,030.90). Learn more about how blocks work.

Hash	0000000000000000000654a1155d0d9467c14ae7c9d5252d00737972fb13cc71
Confirmations	1
Timestamp	2021-10-09 05:32
Height	704137
Miner	Poolin
Number of Transactions	2,837
Difficulty	19,893,045,048,575.13

H(N || M) → a hash with xx-bit of leading zeros

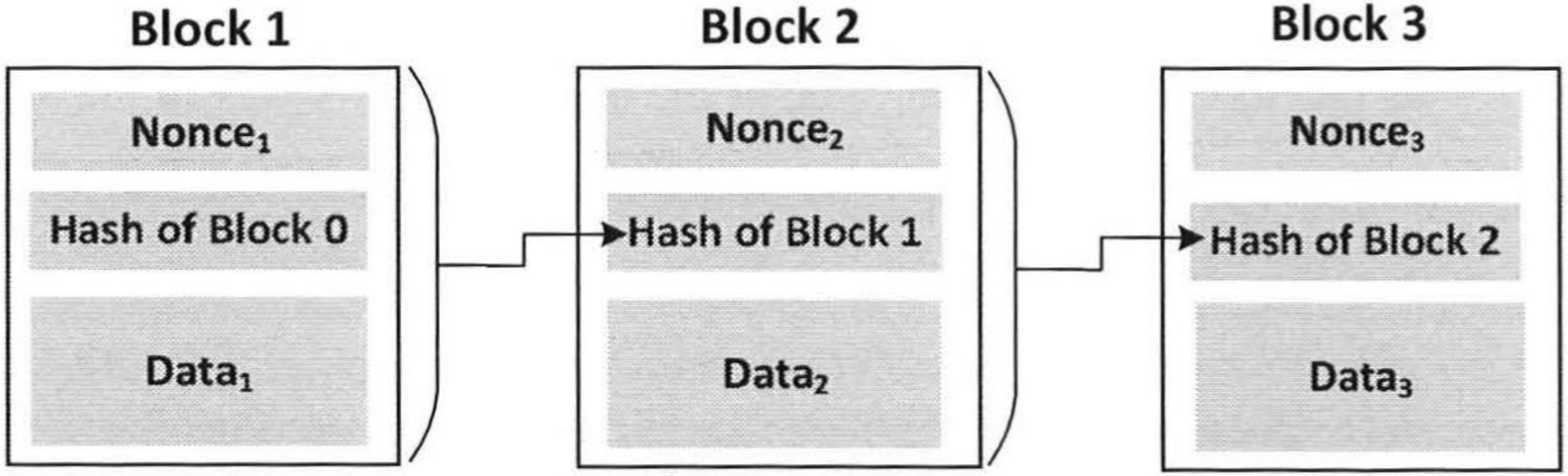


Figure 22.7: Blockchain: a nonce is added to each block

Listing 22.4: Find nonce (`find_nonce.c`)

```
#include <stdio.h>
#include <stdlib.h>
#include <openssl/sha.h>
#include <string.h>

void main()
{
  SHA256_CTX ctx;
  u_int8_t results[SHA256_DIGEST_LENGTH];
  int nonce = 0;
  char *msg = "The data in the block";
  char buf[200];
  int len, i;

while(1) {
  printf("Nonce = %d\n", nonce);
  sprintf(buf, "%d:%s", nonce, msg);
  len = strlen(buf);
  // Compute the SHA256 hash.
  SHA256_Init(&ctx);
  SHA256_Update(&ctx, (u_int8_t *)buf, len);
  SHA256_Final(results, &ctx);
  if (results[0] == 0 && results[1] == 0)
     break;
  else nonce++;
}
```

```
$ gcc find_nonce.c -lcrypto
$ a.out
Nonce = 1
Nonce = 2
... (lines omitted) ...
Nonce = 19678
Nonce = 19679
Nonce = 19680
000037aa9af5901664d5baffdaa257ad7a14c070902aea8f4a6f5d5359ed1f9a

Let us verify it:
$ echo -n "19680:The data in the block" | sha256sum
000037aa9af5901664d5baffdaa257ad7a14c070902aea8f4a6f5d5359ed1f9a  -
```

Adding Incentives and Bitcoin

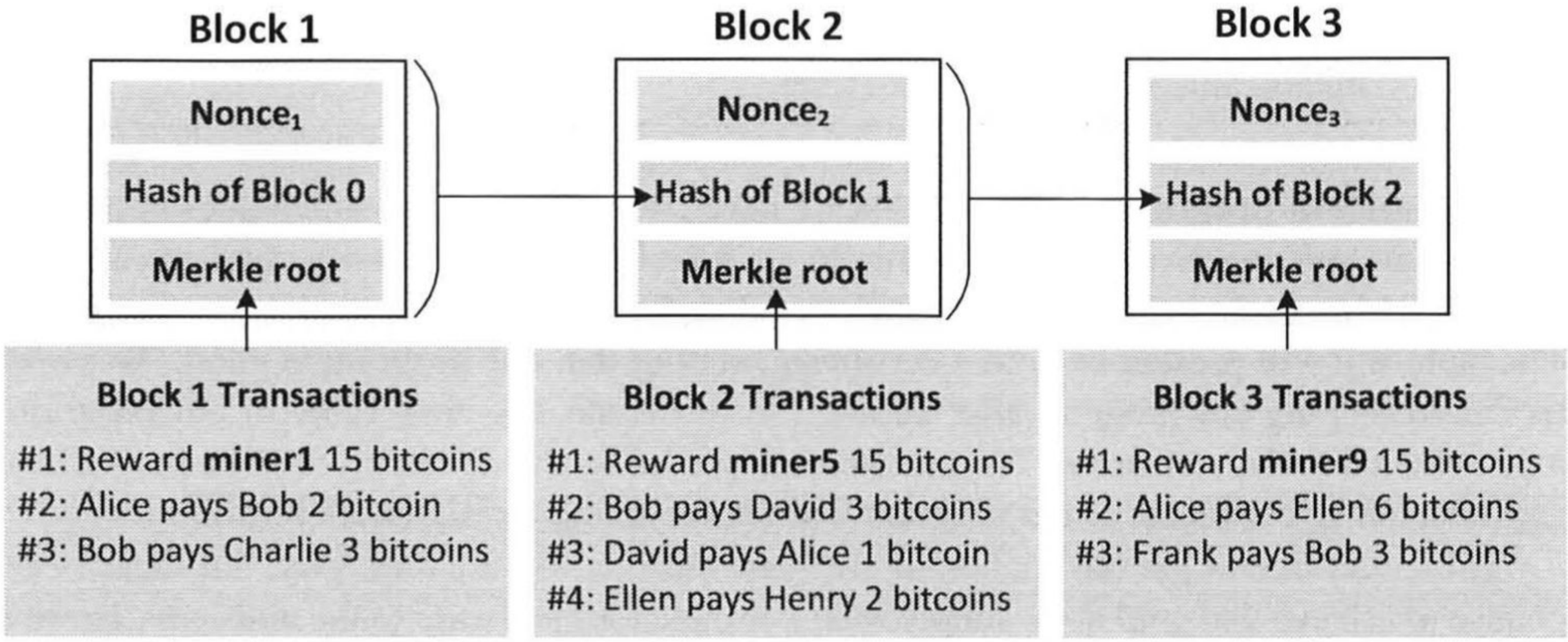


Figure 22.8: Bitcoin blockchain

Hash Collision Attacks

Security Impact of Collision Attacks

- Forging fake public-key certificates
- The integrity of programs
- collision attacks have been found against MD5 and SHA-1

Generating Two Different Files with the Same MDS Hash

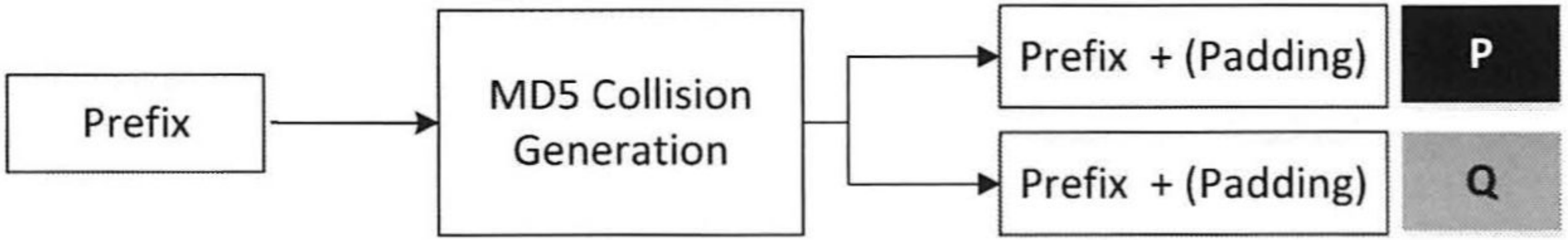


Figure 22.9: MD5 collision generation from a prefix

```
seed@VM:~/.../Lab2$ echo "Message prefix" > prefix.txt
seed@VM:~/.../Lab2$ md5collgen -p prefix.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix.txt'
Using initial value: 0630714724b14391dc74902f303d5b47

Generating first block: ..............................
Generating second block: S01..
Running time: 160.657 s
seed@VM:~/.../Lab2$ md5sum out1.bin
8d8632b8b2d9b7d388024213a59520fa  out1.bin
seed@VM:~/.../Lab2$ md5sum out2.bin
8d8632b8b2d9b7d388024213a59520fa  out2.bin
seed@VM:~/.../Lab2$ sha256sum  out2.bin
219f39f382915a29f60ee84a208c21de31870b1d60da6d2215b4587baab47d47  out2.bin
seed@VM:~/.../Lab2$ sha256sum  out1.bin
ee6cd3ae281e43cf3e6039c5f2c02bb4f6026badebbb82cc8c9c2bfb171ec246  out1.bin
```

```
seed@VM:~/.../Lab2$ echo "This is my extension" > suffix.txt
seed@VM:~/.../Lab2$ cat out1.bin suffix.txt > out1_long.bin
seed@VM:~/.../Lab2$ cat out2.bin suffix.txt > out2_long.bin
seed@VM:~/.../Lab2$ md5sum out1_long.bin
611e75a1c1f34a1704ae8dc69b689b1e  out1_long.bin
seed@VM:~/.../Lab2$ md5sum out2_long.bin
611e75a1c1f34a1704ae8dc69b689b1e  out2_long.bin
```